SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: OPERATING SYSTEMS COURSE CODE: CSA04

COURSE OUTCOME COVERED

CO2: Analyze process management and deadlock handling techniques to improve CPU utilization and system performance (BL3-BL4)

Assessment Tool Weightage: 30%

Dr.J.Velmurugan / Professor
Dept. of CSE

QUESTIONS: 5 **Total Marks:50**

Annexure A

Scenario based

Code of Conduct:

I Hasmath Nisha M(192524513) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Criteria	Weightage (Marks)	Level 4 – Exemplary (Full Marks)	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Total
Understanding of Scenario	**20%**					
Identification of Key Issues	**20%**					
Application of Concepts	**25%**					
Decision Making	**20%**					
Clarity of Response	**15%**					
Total						

Question 1 — CPU Scheduling in a Real-Time Ticket Booking System

A railway ticket booking system experiences heavy load during festival seasons. The system handles three categories of processes:

- P1 – Payment gateway transactions (High priority)
- P2 – Seat availability checks (Medium priority)
- P3 – Ticket confirmation and printing (Low priority)
- Burst times vary from 2 ms to 20 ms.

Recently, customers complained that ticket confirmations are taking too long.

Tasks

1. Identify the most suitable scheduling algorithm (FCFS, SJF, Priority, RR) for this scenario.
2. Justify *why* your chosen method improves CPU utilization and reduces customer wait time.
3. Provide a sample execution order for the above categories.
4. Discuss one drawback of this scheduling method in real-time systems.

(10 Marks)

Question 2 — Deadlock in a Banking Transaction Server

A bank server handles the following operations simultaneously:

- T1: Transfers money between accounts
- T2: Updates passbook entries
- T3: Generates monthly statements

Due to simultaneous locking of Account Database (A), Customer Info Database (B), and Transaction Logs (C), the system enters a state where:

- T1 holds A and requests B
- T2 holds B and requests C
- T3 holds C and requests A

Tasks

1. Draw the resource request situation (text-based RAG).
2. Identify whether a deadlock has occurred and justify your answer.

3. Recommend *one specific technique* (avoidance/prevention/recovery) and explain how it fixes the issue.
4. Suggest a redesign for the locking order to avoid future deadlocks.

(10 Marks)

Question 3 — Mobile OS App Switching Lag

A mobile operating system is designed to support smooth app switching.

But users report that:

- Opening WhatsApp delays the background music player.
- Switching between social media apps feels laggy.
- CPU shows high idle times despite multiple running apps.

The OS currently uses Round Robin (quantum = 20 ms) with 20 background apps active.

Tasks

1. Analyze why the CPU remains idle despite many active processes.
2. Determine whether the time quantum is helping or hurting performance.
3. Recommend a better scheduling method or quantum value, with justification.
4. Explain how the change will reduce switching lag and improve system throughput.

(10 Marks)

Question 4 — Deadlock Risk in an Operating System Update

During a Windows OS update, several modules run in parallel:

- M1: Copy update files
- M2: Install drivers
- M3: Update registry
- M4: Validate system integrity

Resources involved:

- R1 = Disk I/O
- R2 = Kernel lock
- R3 = Configuration lock

Recently logs show:

- M1 holds R1, waiting for R2

- M2 holds R2, waiting for R3
- M3 holds R3, waiting for R1

Tasks

1. Identify whether this sequence can lead to a deadlock.
2. If yes, explain the exact circular wait condition.
3. Propose *Banker's algorithm style* resource allocation to avoid risk.
4. Suggest how OS designers can prioritize modules to improve CPU utilization during update.

(10 Marks)

Question 5 — Cloud Server CPU Utilization Analysis

A cloud hosting provider notices:

- 80% of the time, VM processes wait for network or disk
- Only 20% of the time is spent on CPU
- Each server runs 7 VMs

Using the formula:

$$U = 1 - p^n$$

where p = I/O wait probability

Tasks

1. Calculate CPU utilization.
2. Interpret whether the server is underutilized or optimally loaded.
3. Suggest how increasing or decreasing number of VMs affects throughput.
4. Propose two OS-level strategies for improving CPU utilization in this cloud environment.

(10 Marks)

Question 1: CPU Scheduling in a Real-Time Ticket Booking System

> *P1 = payment transactions (high priority), P2 = seat availability checks (medium), P3 = ticket confirmation and printing (low). Bursts range from 2 ms to 20 ms. Customers complain that ticket confirmations take too long.*

1. Most suitable algorithm

Preemptive Priority Scheduling with aging (with Round Robin among processes of the same priority).

2. Justification

- **Payments must not wait.** P1 is the most time-critical and money-related work. With preemptive priority, a new payment request immediately pre-empts a running P2 or P3, so its response time is minimal.
- **Better than FCFS:** a 20 ms confirmation job in front of a 2 ms payment would delay it (convoy effect).
- **Better than plain SJF:** SJF ignores business importance, and burst lengths (2 to 20 ms) are only estimates in a live system.
- **Better than plain RR:** RR treats a payment and a print job equally, so critical work shares the CPU with unimportant work.
- **Why aging fixes the complaint:** under heavy festival load, P1 and P2 keep arriving, so P3 (confirmation) can starve, which is exactly why confirmations are slow. Aging raises P3's priority gradually while it waits (for example +1 level every 50 ms), so it is guaranteed to run soon without hurting payments much.
- **CPU utilisation:** short bursts finish quickly and the CPU always has a ready process to run, so waiting time drops and the CPU stays busy.

3. Sample execution order

Assume P1 arrives at t = 0 (burst 4 ms), P2 at t = 0 (burst 8 ms), P3 at t = 0 (burst 20 ms), and a second payment request P1' arrives at t = 6 ms (burst 3 ms).

```
| P1 | P2 | P1' | P2 |      P3      |
0    4    6     9    15             35   (time in ms)
```

Process	Arrival	Burst	Completion	Waiting
P1	0	4	4	0
P2	0	8	15	7
P1'	6	3	9	0
P3	0	20	35	15

P2 runs 4 to 6 ms, is pre-empted by P1', and resumes at 9 ms. Payments never wait. With aging, P3 would be promoted if it waited too long, so it cannot be postponed forever.

4. Drawback in real-time systems

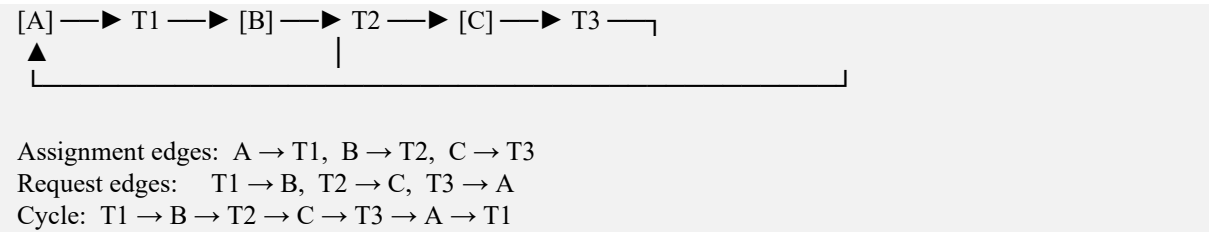
Starvation and priority inversion. Pure priority scheduling can starve low-priority jobs indefinitely, which breaks timing guarantees. Priority inversion can also occur: if P3 holds a lock that P1 needs, P1 is blocked while a medium-priority P2 keeps pre-empting P3. This makes response time unpredictable. It is handled with aging and priority inheritance, and preemption adds context-switch overhead.

Question 2: Deadlock in a Banking Transaction Server

> *T1 holds A and requests B. T2 holds B and requests C. T3 holds C and requests A. A = Account Database, B = Customer Info Database, C = Transaction Logs.*

1. Resource Allocation Graph (text-based)

Edge from resource to process = held by the process. Edge from process to resource = requested by the process.



2. Has a deadlock occurred?

Yes. The graph contains a cycle (T1 → B → T2 → C → T3 → A → T1), and each resource has a single instance. With single-instance resources, a cycle is both necessary and sufficient for deadlock. Each transaction holds one resource and waits for one held by the next, so none can proceed.

All four conditions hold: mutual exclusion (database locks), hold and wait, no pre-emption (locks released only voluntarily), and circular wait.

3. Recommended technique: Deadlock Prevention by resource ordering

Technique: assign a global order to resources, A < B < C, and require every transaction to request locks only in increasing order. This breaks the **circular wait** condition.

- T1 needs A then B: already follows the order.
- T2 needs B then C: already follows the order.
- T3 needs C then A: violates the order. It must lock A first and then C.

With this rule, a transaction holding a higher-numbered resource can never wait for a lower-numbered one, so a cycle cannot form. T1 holds A, so T3 waits at A (holding nothing). T1 gets B, finishes, and the others proceed in turn.

Alternative (recovery): detect the cycle and abort or roll back one transaction (database rollback is natural here), but prevention is cheaper and avoids wasted work.

4. Redesign of the locking order

```
Global order: A (Accounts) < B (Customer Info) < C (Txn Logs)

T1: lock(A) -> lock(B) -> work -> unlock(B) -> unlock(A)
T2: lock(B) -> lock(C) -> work -> unlock(C) -> unlock(B)
T3: lock(A) -> lock(C) -> work -> unlock(C) -> unlock(A)   // A first
```

Further improvements: hold locks for the shortest time, use try-lock with timeout and retry, and let T3 (monthly statements, read-only) read a consistent snapshot instead of taking write locks, so it never blocks T1 and T2.

Question 3: Mobile OS App Switching Lag

> *Round Robin with quantum = 20 ms and 20 background apps. Symptoms: WhatsApp delays the music player, switching between social apps feels laggy, and the CPU shows high idle time.*

1. Why the CPU is idle despite many active apps

- **Most apps are blocked, not ready.** Background apps spend their time waiting for network responses, timers, sensors, push notifications or user input, so only a few are in the ready queue at any moment.
- **Apps are I/O-bound with short bursts:** they run for a few ms, then block again, so the CPU is idle between bursts even though 20 apps exist.
- **No priority awareness:** Round Robin treats the music player, WhatsApp and a idle background sync identically, so the work that matters is not given the CPU first.

So "many running apps" does not mean "many runnable processes". Idle CPU and lag happen together because the CPU is not being given to the right process at the right time.

2. Is the 20 ms quantum helping or hurting?

It is hurting. If all 20 apps are ready, a given app may wait up to 19 × 20 ms = 380 ms for its turn. That is far above the roughly 100 ms users perceive as instant, and it is long enough for audio buffers to run dry, so the music stutters. Most app bursts are far shorter than 20 ms, so the long quantum gives no efficiency benefit for them, while interactive response suffers.

3. Recommended change

Use a Multilevel Feedback Queue (MLFQ) with priority classes and a smaller quantum (about 5 to 10 ms) for interactive queues.

Queue	Apps	Policy
Q0: real-time / audio	Music player, calls	Highest priority, small quantum, pre-empts others
Q1: foreground / interactive	App the user is using, WhatsApp	Quantum 5 to 10 ms, RR
Q2: background	Social media sync, updates	Larger quantum, low priority, throttled

Apps move between queues based on behaviour (I/O-bound apps stay high, CPU-hogging ones drop), with aging so background apps still get CPU. Idle background apps can be frozen or batched by the OS.

4. How this reduces lag and improves throughput

- The foreground app and the music player get the CPU within a few ms instead of up to 380 ms, so app switching feels immediate and audio stays smooth.
- Short quanta for interactive queues cut response time; I/O-bound apps that block early keep their high priority, so they run as soon as their I/O completes.
- Background apps no longer steal slices from important ones, and the CPU is used for useful work, which raises throughput.
- Keeping the quantum above 5 ms avoids excessive context-switch overhead.

Question 4: Deadlock Risk in an Operating System Update

> *Modules M1 (copy files), M2 (install drivers), M3 (update registry), M4 (validate integrity). Resources: R1 = Disk I/O, R2 = Kernel lock, R3 = Configuration lock. M1 holds R1 and waits for R2, M2 holds R2 and waits for R3, M3 holds R3 and waits for R1.*

1. Can this sequence lead to deadlock?

Yes, it is already a deadlock if each resource has a single instance, because the wait-for relationships form a closed loop.

2. Exact circular wait

```
M1 ──waits for R2──► M2 ──waits for R3──► M3 ──waits for R1──► M1
(R2 is held by M2)    (R3 is held by M3)    (R1 is held by M1)
```

M1 → M2 → M3 → M1 is a cycle. Each module holds one resource and waits for a resource held by the next module in the cycle, so none can finish or release anything. M4 is not in the cycle, but it is blocked if it needs any of these resources.

3. Banker's algorithm style allocation

Each module declares its maximum need in advance. Assume single instances (available at start: R1 = 1, R2 = 1, R3 = 1) and M4 needs only R1 for validation.

Module	Max need	Holds (in the logs)	Still needs
M1	R1, R2	R1	R2
M2	R2, R3	R2	R3
M3	R3, R1	R3	R1
M4	R1	none	R1

Safety check of the logged state: Available = nothing. M1 needs R2 (held by M2), M2 needs R3 (held by M3), M3 needs R1 (held by M1). No module can complete, so no safe sequence exists, and the state is **unsafe**.

How the algorithm avoids it: before granting any request, the OS pretends to grant it and checks whether a safe sequence still exists.

1. M1 requests R1: grant (M1 can later finish with R2). State safe.
2. M2 requests R2: grant. Remaining free: R3. M2 can take R3, finish and release R2 and R3. State safe.
3. **M3 requests R3: DENY.** Granting it would leave nothing available and nobody able to finish (unsafe state). M3 waits.
4. M2 gets R3, finishes, releases R2 and R3. M1 gets R2, finishes, releases R1 and R2. M3 then gets R3 and R1 and finishes. M4 uses R1 last.

Safe sequence: M2 → M1 → M3 → M4. The deadlock never forms.

4. Prioritising modules to improve CPU utilisation

- **Follow the natural dependency order:** copy files (M1) → install drivers (M2) → update registry (M3) → validate (M4), and give higher priority to modules on the critical path or holding scarce locks, so they release resources sooner.
- **Overlap I/O-bound and CPU-bound work:** M1 is disk-heavy and M4 is compute-heavy, so schedule them together so the CPU is busy while the disk is busy.
- **Use ordered locking (R1 < R2 < R3)** so modules never wait in a cycle, and use priority scheduling with aging so low-priority modules still finish.
- **Use asynchronous disk I/O** so a module waiting on the disk does not stall the CPU.

Question 5: Cloud Server CPU Utilisation Analysis

> *80% of the time VMs wait for network or disk, 20% of the time is spent on CPU, each server runs 7 VMs. Formula: U = 1 − p^n, where p = I/O wait probability and n = number of VMs (processes).*

1. CPU utilisation

```
p = 0.8,  n = 7
U = 1 - p^n = 1 - (0.8)^7
 = 1 - 0.2097
 = 0.7903  ->  about 79%
```

The CPU is idle only when all 7 VMs are waiting at the same time, which has probability 0.8^7 ≈ 21%.

2. Interpretation

The server is moderately loaded and slightly under-utilised. About 79% utilisation is decent, but roughly 21% of CPU capacity is wasted. A well-run cloud server normally aims for about 90% or more without overload, so there is room to add VMs.

3. Effect of increasing or decreasing the number of VMs

Number of VMs (n)	p^n (CPU idle)	Utilisation U	Comment
1	0.8000	20.0%	Very low
3	0.5120	48.8%	Low
5	0.3277	67.2%	Moderate
7 (current)	0.2097	79.0%	Fairly good
10	0.1074	89.3%	Good
11	0.0859	91.4%	Reaches 90%
14	0.0440	95.6%	Reaches 95%
20	0.0115	98.8%	Near-saturated

- **Increasing VMs** raises utilisation and total throughput, but with diminishing returns: going from 7 to 11 VMs adds about 12 points, while going from 14 to 20 adds only about 3 points. About 11 VMs reach 90% and about 14 reach 95%.
- **Limits:** the formula assumes VMs wait independently and ignores overheads. In practice too many VMs cause memory pressure and swapping, more context switching, cache and I/O contention, and slower response per VM.
- **Decreasing VMs** lowers utilisation (e.g. 3 VMs: 48.8%), wastes hardware and cost, though each VM sees better latency.

4. Two OS-level strategies to improve utilisation

5. **Increase the degree of multiprogramming with admission control.** Pack more VMs per server (CPU and memory overcommit with ballooning and page sharing), and use a hypervisor scheduler that mixes CPU-bound and I/O-bound VMs. Monitor memory and stop admitting VMs before thrashing begins.
6. **Reduce I/O wait with asynchronous I/O and faster I/O paths.** Use non-blocking I/O, caching and read-ahead, paravirtualised (virtio) drivers, SSD or network-attached storage with I/O scheduling, so VMs spend less of their time waiting and the CPU is idle less often. A lower p raises U for the same n.

RUBRICS FOR CALCULATION - Scenario based assessment

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand